Data.Functor.Contravariant: https://hackage.haskell.org/package/base/docs/Data-Functor-Contravariant.html

```
class Contravariant (f :: Type -> Type) where
 contramap :: (a' -> a) -> f a -> f a'
 (>$) :: b -> f b -> f a

(>$<) :: (a' -> a) -> f a -> f a'
(>$$<) :: f a -> (a' -> a) -> f a'
($<) :: f b -> b -> f a
```

Instances

```
instance Contravariant (Op a)
instance Contravariant Predicate
instance Contravariant Comparison
instance Contravariant Equivalent
```

Contravariant
Dual of Functor
i.e.
Functor = map of function **range**
Contravariant = map of function **domain**

i.e. for any f : a → b, u : a' → a, v : b → b'

v (f (u (x))) v = contravariant
u = functor

```
---- Basic
--
>$< :: (a' -> a) -> f a -> f a'
>$$< :: f a -> (a' -> a) -> f a'
>$ :: b -> f b -> f a
$< :: f b -> b -> f a
g >$< Op f                    -- f(g(x))
(+1) >$< Op (*10)             -- 10(x+1)
getOp ((+1) >$< Op (*10)) 1   -- 20
getOp ((+1) >$< Op (*10)) 10  -- 110

a >$ Op f                     -- f(x) = f(a) i.e.
constant function
2 >$ Op (*10)                 -- f(x) = 20
getOp (2 >$ Op (*10)) 1       -- 20
getOp (2 >$ Op (*10)) 10      -- 20
```

Profunctor (external class)
Ref: https://hackage.haskell.org/package/profunctors

```
class Profunctor p where
 {# MINIMAL lmap,rmap | dimap #}
```

```
 lmap :: (a' -> a) -> p a b -> p a' b
 rmap :: (b -> b') -> p a b -> p a b'
 dimap :: (a' -> a) -> (b -> b') -> p a b -> p a' b'

class Profunctor p => Choice p where
 {# MINIMAL right'|left' #}
 right' :: p a b -> p (Either c a) (Either c b)
 left' :: p a b -> p (Either a c) (Either b c)

class Profunctor p => Strong p where
 {# MINIMAL first'|second' #}
 first' :: p a b -> p (a, c) -> (b, c)
 second' :: p a b -> p (c, a) -> (c, b)
```

Instances

```
instance Monad m => Profunctor (Kliesli m)
instance Profunctor (->)
```

Profunctor
Contravariant + functor i.e. can map both domain and range.

Choice

right' :: p a b -> p (Either c a) (Either c b)	(c OR a) ↦ (c OR f(a))
left' :: p a b -> p (Either a c) (Either a b)	(a OR c) ↦ (f(a) OR c)

Strong

first' :: p a b -> p (a, c) (b, c)	(a, c) ↦ (f(a), c)
second' :: p a b -> p (c, a) (c, b)	(c, a) ↦ (c, f(a))